RedSens

Guía de estudio de M1 lineal

FEDE · Grupo 3 · Taller de Algoritmos y Estructura de Datos II

Esta guía explica la tabla hash que implementamos, cómo ejecutarla y cómo defender sus decisiones. Incluye cada línea de los dos archivos de producción propios, de la prueba automatizada y de la práctica manual. Las líneas coinciden con los archivos que acompañan esta entrega.

Tu responsabilidad: base de TablaHash, exploración lineal, inserción, búsqueda, eliminación perezosa y mediciones. La exploración cuadrática y el rehashing son la siguiente tarea del equipo.

Estado validado: 15.205 verificaciones de M1, 57 del ABB y todas las comprobaciones del AVL correctas. La práctica de JShell también se ejecutó. La capacidad de la tabla sigue siendo fija.

Ruta de lectura

1. Entender el programa y tu entrega.
2. Entender la estructura y sus alternativas.
3. Seguir una traza a mano.
4. Compilar, ejecutar y experimentar.
5. Coordinar la continuación y el Pull Request.
6. Estudiar el código línea por línea.
7. Practicar preguntas de exposición.

Parámetros del grupo

Parámetro	Valor
Semilla Random	3026
Capacidad inicial de M1	17
Base de cuadrilla de M3	San Francisco
M de Josephus	3

Es material de estudio. El informe técnico final de hasta 15 páginas y la bitácora de IA son entregables distintos. La explicación de aquí debe practicarse con tus propias palabras.

1 Qué hace RedSens y qué hicimos

RedSens representa una red provincial de estaciones de medición. Cada estación tiene código, número, nombre, localidad, tipo, batería y días sin visita. El programa final debe permitir encontrarlas, listar las críticas, planificar mantenimiento, comprimir mediciones y distribuir guardias.

La necesidad de M1 es localizar una estación por su código sin recorrer todos los registros. Si la central recibe CBA-0013, la tabla devuelve el objeto Estacion correspondiente. Ese objeto contiene sus datos; la tabla no vuelve a generarlos.

Módulo o persona	Relación con tu parte
LUCA · M6	Genera objetos Estacion reproducibles con semilla 3026. M1 debe recibirlos por insertar; las pruebas actuales usan datos propios para poder ejecutarse por separado.
FEDE · M1 lineal	Construye el registro común y resuelve colisiones avanzando casilla por casilla.
Quinto integrante · M1 cuadrática	Agrega otra estrategia sobre la misma tabla y desarrolla carga, rehashing y comparación. En el roadmap figura como integrante 4: es la quinta persona del equipo, no TOBI.
TOBI · ABB y BENJA · AVL	Indexan las mismas estaciones por numero. Permiten orden, rangos y críticas. M1 usa codigo y sirve para acceso directo promedio.
M3	Recibe las estaciones críticas del índice M2 y relaciona sus localidades con el grafo. No reemplaza el registro M1.
M4 y M5	Huffman comprime mediciones; Josephus organiza técnicos. No llaman necesariamente a la tabla en esta primera versión.
Integración y menú	Debe insertar una misma estación en hash y AVL, y mantener ambos coherentes al dar altas o bajas. Esta coordinación todavía no está implementada.

Qué existe hoy y qué falta

En main ya existen el modelo Estacion, ABB, AVL, sus pruebas y el experimento de M2. Main solo muestra “RedSens iniciado”: todavía no es el menú final. Esta rama agrega el registro lineal, sus pruebas, una práctica manual y documentación.

No implementamos cuadrática, rehashing, el experimento comparativo completo de M1 ni la integración del generador y el menú. Una tabla de capacidad 17 no crece: el intento de insertar cuando no hay casillas disponibles falla de forma controlada sin perder datos.

2 La estructura elegida y por qué

Elegimos una **tabla hash propia con direccionamiento abierto y exploración lineal**. Se usa un arreglo de casillas. Una función hash transforma el código en una posición inicial; si está ocupada por otra clave, se prueban posiciones consecutivas, volviendo al inicio con el operador módulo.

En las lecturas se llama también hash cerrado a este direccionamiento abierto. No hay listas enlazadas de estaciones dentro de cada posición: los registros ocupan otras casillas del mismo arreglo.

Tres estados distintos

Representación	Significado
casillas[i] == null	Nunca ocupada. Una búsqueda puede terminar aquí.
Casilla existente con estacion != null	Ocupada por una estación activa.
Casilla existente con estacion == null	Borrada lógicamente. La búsqueda debe seguir.

Hay dos niveles: el arreglo contiene referencias a Casilla; cada Casilla puede contener una referencia a Estacion. Al borrar hacemos casillas[posicion].estacion = null. La Casilla continúa existiendo y conserva la marca. Si borráramos la Casilla completa, podríamos cortar una búsqueda y perder acceso a registros posteriores de la cadena.

La función hash

```
h = 0
h = (31 * h + caracter) % capacidad  // repetir para cada carácter
posicion = (h + intento) % capacidad
```

Se procesan todos los caracteres de CBA-NNNN. charAt obtiene un carácter y Java lo usa numéricamente. El prefijo CBA- es común: la diferencia entre claves viene de los dígitos. La base 31 hace que importe su orden. No es un hash criptográfico ni garantiza ausencia de colisiones. El 31L fuerza aritmética long antes de reducir, para evitar desbordamiento de int.

Costos y límites

Con códigos de longitud fija y carga controlada, se espera O(1) promedio para insertar, buscar y eliminar. En el peor caso se recorren m casillas: O(m), donde m es la capacidad. La memoria es O(m). El borrado puede ser costoso por la búsqueda que lo precede; marcar la posición encontrada cuesta O(1).

El factor de carga es n/m: registros activos divididos por capacidad. Al borrar baja n, pero siguen existiendo marcas que pueden alargar búsquedas. Por eso el otro integrante deberá controlar tanto crecimiento como acumulación de marcas. El rehashing redistribuye las claves; no basta copiar el arreglo, porque el módulo cambia al cambiar m.

3 Qué pasaría al cambiar la estructura

Alternativa	Qué cambiaría
Lista o arreglo sin índice	Buscar por código exige recorrer hasta n estaciones: O(n). Es sencillo, pero pierde el objetivo de acceso constante promedio.
ABB	Permite orden y rangos. En promedio puede buscar en O(log n), pero degenera a O(n) con entradas ordenadas. No reemplaza la tabla propia exigida por M1.
AVL	Garantiza O(log n) para búsqueda y actualización, y permite rangos. Requiere alturas y rotaciones. Es el complemento M2, no la solución requerida para M1.
HashMap de Java	Ocultaría la implementación que la materia exige desarrollar. El enunciado lo prohíbe como reemplazo del registro M1.
Encadenamiento	Cada posición puede contener una lista de registros. Cambian eliminación, memoria y mediciones. Es una variante opcional del enunciado, no sustituye las dos exploraciones obligatorias.
Exploración cuadrática	Conserva el arreglo, las operaciones y el hash. Cambia el cálculo de las posiciones. Reduce agrupamiento primario, pero requiere cuidar cobertura y umbral de carga.
Arreglo directo por numero	Con este dominio fijo 0..9999 podría reservar 10.000 lugares y acceder por índice en O(1) peor caso. Consume el espacio del dominio y no cumple el objetivo de practicar la tabla hash y sus colisiones.

Por qué lineal primero

Es sencilla de trazar, recorre todas las casillas y deja clara la diferencia entre casilla nueva, ocupada y borrada. Su desventaja es el agrupamiento primario: bloques consecutivos de posiciones ocupadas crecen y fuerzan recorridos más largos. La comparación con cuadrática debe medir esa diferencia con los mismos datos y la misma definición de colisión.

La extensión que dejamos

TablaHash no calcula directamente “inicio + intento” dentro de sus operaciones: delega en estrategia.posicion. El enum EstrategiaExploracion contiene LINEAL y una operación abstracta. El compañero agregará CUADRATICA sin copiar insertar, buscar ni eliminar.

Para rehashing también deberá cambiar el arreglo casillas, hoy final, y agregar el crecimiento. final impide reasignar la referencia al arreglo, no impide cambiar sus elementos. La tabla se entrega deliberadamente con capacidad fija porque crecimiento es parte de la otra responsabilidad.

Con desplazamientos i*i y capacidad prima impar, la exploración cuadrática no recorre todas las posiciones. Una carga activa estrictamente inferior a 0,5 asegura disponibilidad dentro de las posiciones alcanzables. El umbral de lineal podría ser 0,7; ambos valores siguen siendo propuestas que el responsable deberá justificar e implementar.

4 Traza manual con tres estaciones

La práctica usa capacidad 17, la inicial del grupo. Los códigos CBA-0000, CBA-0013 y CBA-0026 producen la misma posición inicial: 12. No es una suposición: puede verificarse con la función implementada.

Inserción	Recorrido y resultado	Sondeos	Colisiones
CBA-0000	12 vacía: guardar en 12	1	0
CBA-0013	12 ocupada, 13 vacía: guardar en 13	2	1
CBA-0026	12 ocupada, 13 ocupada, 14 vacía: guardar en 14	3	2
Totales	Tres registros activos de capacidad 17	6	3

```
Índice:      12           13           14          15
Estado:      CBA-0000     CBA-0013     CBA-0026    nunca usada
Secuencia:   12  ->       13  ->       14  ->       15
```

Una colisión cuenta si la casilla está activa con un código diferente. Un sondeo cuenta cada lectura de una casilla: también las vacías y borradas. La coincidencia con el código buscado no es colisión. La escritura en una posición ya examinada no suma otro sondeo.

Borrar CBA-0000 y buscar CBA-0026

```
Índice:      12           13           14
Estado:      BORRADA      CBA-0013     CBA-0026
```

Borrar 0000 busca la clave, encuentra la casilla 12 y elimina su referencia a Estacion; quedan dos estaciones activas. Al buscar 0026 se lee la marca 12 y se continúa; se lee 0013 en 13, que es una colisión; se encuentra 0026 en 14. Resultado: **3 sondeos y 1 colisión**.

Si hubiéramos puesto casillas[12] = null, la búsqueda habría concluido incorrectamente en la primera posición. Este es el motivo concreto del borrado perezoso.

Reinsertar CBA-0000

Insertar recuerda la marca 12, pero no escribe inmediatamente: debe descartar que 0000 exista más adelante. Lee 12, 13, 14 y 15. En 15 encuentra una casilla nunca usada y guarda en la primera marca, 12. Resultado: **4 sondeos y 2 colisiones**. El tamaño vuelve a 3 y la carga es 3/17 = 0,1764705882.

Si se intenta insertar otra vez 0026 mientras 12 está borrada, debe rechazarse el duplicado existente en 14. Por eso no se puede ocupar la primera marca sin terminar la comprobación de duplicados.

5 Cómo compilar y ejecutar tu parte

Abrí PowerShell en la carpeta RedSens. Tener Java para ejecutar no alcanza: necesitás un JDK, que trae javac y jshell. En esta computadora se validó con OpenJDK 23 y compatibilidad Java 17.

Paso 1 Comprobar herramientas

```
java -version
javac -version
jshell --version
```

Si no se reconocen, en esta computadora podés agregar temporalmente el JDK al PATH de esa consola. No hace falta modificar configuraciones permanentes:

```
$env:Path = "C:\Program Files\Java\OpenJDK-23\bin;" + $env:Path
```

Paso 2 Compilar desde la raíz

```
$fuentes = @(Get-ChildItem src/main/java,src/test/java -Recurse -Filter *.java |
    ForEach-Object { $_.FullName })
javac -encoding UTF-8 --release 17 -d out @fuentes
```

Get-ChildItem reúne los archivos .java. UTF-8 conserva tildes en fuentes. --release 17 comprueba compatibilidad con Java 17. -d out guarda los .class compilados fuera de src. @fuentes pasa cada ruta como argumento, incluso si contiene espacios.

Paso 3 Ejecutar pruebas

```
java -cp out redsens.registro.PruebaTablaHash
java -cp out redsens.indice.PruebaABB
java -cp out redsens.indice.ComprobadorArbolAVL
```

El primer comando debe terminar con “Tabla hash lineal: 15205 verificaciones OK”. Si una condición falla se lanza AssertionError. No se necesita -ea: las pruebas llaman a un método verificar propio, no a la palabra reservada assert.

Paso 4 Practicar en una consola interactiva

```
jshell --class-path out
```

Copiá las líneas de ejemplos/m1-manual.jsh una por una, respetando el orden. No copies números de línea del PDF ni el prefijo jshell>. Para ejecutar todo el ejemplo de una vez, salí con /exit y corré:

```
jshell --class-path out ejemplos/m1-manual.jsh
```

La sesión queda abierta. Las variables tabla, e0, e13 y e26 siguen disponibles. jshell no exige una clase con main para estas expresiones. En la consola normal, java -cp out redsens.Main todavía solo muestra el mensaje de inicio.

6 Experimentos que podés hacer a mano

Resultado esperado de la práctica incluida

```
Inserción de 0026: 3 sondeos, 2 colisiones
Acumulados: 6 sondeos, 3 colisiones
Tamaño: 3; capacidad: 17
Buscar 0026 tras borrar 0000: Estación veintiséis
Búsqueda de 0026: 3 sondeos, 1 colisiones
Buscar 0000 borrada: null
Reinsertar 0000: 4 sondeos, 2 colisiones
Factor de carga: 0.17647058823529413
Sondeos tras reiniciar: 0
```

Rechazar un duplicado

```
tabla.insertar(e13);
tabla.tamanio();
```

Debe aparecer IllegalArgumentException y el tamaño seguir en 3. Una excepción en una instrucción de JShell no cierra la sesión; podés consultar el estado después.

Buscar y eliminar una clave ausente

```
tabla.buscar("CBA-0099");
tabla.eliminar("CBA-0099");
```

La búsqueda devuelve null; eliminar lanza IllegalArgumentException. Ambas operaciones siguen su recorrido normal antes de concluir ausencia. Un código inválido como CBA-99 se rechaza antes de sondear y no sirve para medir búsquedas fallidas.

Observar una tabla llena sin cargar diecisiete objetos

```
TablaHash pequena = new TablaHash(1);
pequena.insertar(e0);
pequena.insertar(e13);
pequena.eliminar("CBA-0000");
pequena.insertar(e13);
pequena.buscar("CBA-0013").getNumero();
```

La segunda inserción lanza IllegalStateException porque no hay lugar. Después del borrado se reutiliza la marca y el resultado final es 13. Capacidad 1 es un caso de prueba; no cambia la capacidad inicial 17 asignada al grupo.

Interpretar los contadores sin confundir operaciones

getSondeosUltimaOperacion cambia con cada insertar, buscar o eliminar. Si querés estudiar una operación, consultá sus contadores inmediatamente. Los getters no reinician ni incrementan mediciones. reiniciarContadores pone a cero última operación y acumulados, sin borrar estaciones.

Si la consola muestra caracteres raros, la codificación de salida de la terminal puede diferir de UTF-8; eso no significa que el código Java esté mal. Los comandos de compilación sí especifican UTF-8 para leer correctamente las fuentes.

7 Entrega al compañero y Pull Request

Una rama en GitHub permite que el compañero descargue el trabajo. Un Pull Request propone integrarlo en main y permite revisarlo. Publicar la rama no modifica main. El PR de esta entrega debe revisarse antes de fusionarse; no se considera terminado M1 completo por aprobar la parte lineal.

Si el PR ya fue integrado

```
git switch main
git pull --ff-only origin main
git switch -c feat/tabla-hash-cuadratica
```

El compañero parte de main actualizado y desarrolla la continuación en su propia rama. Los comandos suponen que no tiene cambios locales que impidan cambiar de rama.

Si quiere empezar mientras el PR sigue abierto

```
git fetch origin
git switch -c feat/tabla-hash-cuadratica origin/codex/tabla-hash-lineal
```

Así conserva tus archivos como base. Una vez integrado el primer PR, deberá actualizar su rama con main según la política del equipo. Hasta entonces, un PR de su rama contra main también puede mostrar los cambios de esta entrega: debe indicar esa dependencia.

Tareas concretas que continúan

1. Agregar CUADRATICA en EstrategiaExploracion y justificar fórmula y cobertura.
2. Definir umbrales distintos si corresponde; detectar cuándo crecer.
3. Permitir reasignar casillas y crear capacidad prima desde el doble de la actual.
4. Reinsertar solo estaciones activas y descartar marcas.
5. Contar rehashings y separar trabajo interno de mediciones de usuario.
6. Probar conservación de datos, duplicados y borrados en ambas estrategias.
7. Ejecutar el experimento con 10.000 estaciones de LUCA y comparar colisiones, sondeos exitosos/fallidos y rehashings.

Los códigos válidos posibles son exactamente 10.000. Al insertarlos todos no queda un código válido ausente. Para las búsquedas fallidas, el equipo debe documentar un escenario adicional, como borrar una muestra después de medir la carga completa y consultar esos códigos. No hay que inventar claves inválidas para obtener promedios artificiales de cero sondeos.

Qué debe comprobar la integración del sistema

Hash y AVL deben contener los mismos registros activos. El menú o servicio coordinador debe validar y gestionar fallos para que una inserción o baja no actualice solo uno de ellos. M1 no actualiza el AVL por su cuenta; tampoco genera datos ni planifica rutas.

8 EstrategiaExploracion línea por línea

src\main\java\redsens\registro\EstrategiaExploracion.java

Cada fila corresponde a una línea física del archivo. Las líneas vacías, comentarios y llaves se conservan para que los números coincidan. Leé primero el código y después explicá su efecto sin mirar la tercera columna.

Línea	Código original	Qué hace y por qué
1	`package redsens.registro;`	Ubica el enum en el mismo paquete que TablaHash.
2		Separación visual; no ejecuta instrucciones.
3	`/** Calcula la posición del intento i, empezando en i = 0. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
4	`public enum EstrategiaExploracion {`	Declara un conjunto de estrategias posibles con comportamiento asociado.
5	`    LINEAL {`	Define LINEAL como una constante con su implementación de posicion.
6	`        @Override`	Indica al compilador que está implementando una operación declarada por el enum.
7	`        public int posicion(int inicio, int intento, int capacidad) {`	Recibe posición hash, intento actual y capacidad; devuelve un índice.
8	`            return (int) (((long) inicio + intento) % capacidad);`	Avanza intento posiciones y aplica módulo para volver al inicio. El cast a long evita desbordar la suma antes del módulo.
9	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
10	`    };`	Cierra el cuerpo de LINEAL y termina la lista de constantes; antes del punto y coma podrá agregarse CUADRATICA.
11		Separación visual; no ejecuta instrucciones.
12	`    /**`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
13	`     * Punto común para incorporar CUADRATICA sin duplicar las operaciones.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
14	`     * La estrategia debe mantener el intento 0 en la posición inicial.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
15	`     */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
16	`    public abstract int posicion(int inicio, int intento, int capacidad);`	Contrato abstracto: cada constante debe definir cómo se calcula una posición.
17	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

17 líneas explicadas. SHA-256: 2849ba9c0f9be270fb13d964b59935af18644cf85788091d1126183cde6af7d6

9 TablaHash línea por línea

src\main\java\redsens\registro\TablaHash.java

Cada fila corresponde a una línea física del archivo. Las líneas vacías, comentarios y llaves se conservan para que los números coincidan. Leé primero el código y después explicá su efecto sin mirar la tercera columna.

Línea	Código original	Qué hace y por qué
1	`package redsens.registro;`	Declara el paquete de M1; coincide con la carpeta redsens/registro.
2		Separación visual; no ejecuta instrucciones.
3	`import redsens.modelo.Estacion;`	Importa el modelo común. La tabla almacena Estacion, no inventa otro modelo.
4		Separación visual; no ejecuta instrucciones.
5	`/**`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
6	`* Registro propio con direccionamiento abierto y eliminaci ón perezosa.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
7	`* Esta primera entrega implementa LINEAL con capacidad fij a. La ampliación`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
8	`* automática y CUADRATICA corresponden a la segunda parte de M1.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
9	`*/`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
10	`public class TablaHash {`	Expone la clase que usarán el generador, el menú y las pruebas.
11	`/** null en el arreglo significa que la casilla nunca e stuvo ocupada. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
12	`private static final class Casilla {`	Define una clase interna de casilla. private la oculta; static no liga cada casilla a una instancia externa; final impide heredar de ella.
13	`// Una casilla existente con estacion == null es un a marca de borrado.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
14	`private Estacion estacion;`	Referencia mutable a la estación. null dentro de una casilla existente significa borrada.
15		Separación visual; no ejecuta instrucciones.
16	`private Casilla(Estacion estacion) {`	Constructor privado de una casilla que recibe el objeto a guardar.
17	`this.estacion = estacion;`	Asigna el parámetro al atributo de esa casilla; this distingue ambos nombres.
18	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
19	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
20		Separación visual; no ejecuta instrucciones.
21	`private final Casilla[] casillas;`	Arreglo de referencias a casillas. final fija la referencia al arreglo; sus posiciones sí se pueden modificar.

Línea	Código original	Qué hace y por qué
22	`    private final EstrategiaExploracion estrategia;`	Conserva la estrategia elegida al construir; no se cambia con cada operación.
23	`    private int tamanio;`	Cuenta estaciones activas, no capacidad ni marcas.
24	`    private int sondeosUltimaOperacion;`	Número de casillas leídas por la última operación pública de datos.
25	`    private int colisionesUltimaOperacion;`	Cantidad de claves activas diferentes encontradas en la última operación.
26	`    private long sondeosTotales;`	Acumulado de sondeos; long admite más mediciones que int.
27	`    private long colisionesTotales;`	Acumulado de colisiones desde la creación o el último reinicio.
28		Separación visual; no ejecuta instrucciones.
29	`    public TablaHash(int capacidad) {`	Constructor abreviado: el usuario solo elige capacidad.
30	`        this(capacidad, EstrategiaExploracion.LINEAL);`	Delega en el constructor completo usando LINEAL. this(...) aquí llama otro constructor, no accede a un atributo.
31	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
32		Separación visual; no ejecuta instrucciones.
33	`    public TablaHash(int capacidad, EstrategiaExploracion estrategia) {`	Constructor completo: permite seleccionar estrategia al crear la tabla.
34	`        if (capacidad <= 0) {`	Evita arreglos vacíos y capacidades negativas.
35	`            throw new IllegalArgumentException("La capacidad debe ser positiva");`	Rechaza el argumento con una excepción que explica el problema.
36	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
37	`        if (estrategia == null) {`	Comprueba que haya una estrategia concreta.
38	`            throw new IllegalArgumentException("La estrategia no puede ser nula");`	Rechaza null antes de usar estrategia.posicion.
39	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
40	`        this.casillas = new Casilla[capacidad];`	Reserva capacidad referencias, inicialmente todas null. No crea las Casilla individuales.
41	`        this.estrategia = estrategia;`	Guarda la estrategia recibida en el atributo.
42	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
43		Separación visual; no ejecuta instrucciones.
44	`    /** Rechaza duplicados; una tabla llena no pierde ni reemplaza estaciones. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
45	`    public void insertar(Estacion estacion) {`	Inserta una estación; void indica que no devuelve un resultado.
46	`        iniciarOperacion();`	Reinicia solo los contadores de la última operación, no los acumulados.
47	`        if (estacion == null) {`	Comprueba que haya un objeto para insertar antes de pedir su código.

Línea	Código original	Qué hace y por qué
48	`            throw new IllegalArgumentException("La estación no puede ser nula");`	Rechaza null sin leer casillas.
49	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
50	`        int inicio = hash(estacion.getCodigo());`	Calcula la posición inicial usando el código validado por el constructor de Estacion.
51	`        int primeraBorrada = -1;`	Usa -1 como señal de que aún no encontró una marca reutilizable.
52	`        for (int intento = 0; intento < casillas.length; intento++) {`	Prueba a lo sumo capacidad intentos. Evita un ciclo infinito con tabla llena o borrada.
53	`            int posicion = estrategia.posicion(inicio, intento, casillas.length);`	Delega la posición del intento en la estrategia; ese es el punto de extensión.
54	`            Casilla casilla = sondear(posicion);`	Lee la casilla y suma un sondeo a ambos contadores.
55	`            if (casilla == null) {`	Distingue una posición nunca ocupada de una marca de borrado.
56	`                ocupar(primeraBorrada >= 0 ? primeraBorrada : posicion, estacion);`	Operador ternario: si recordó una marca, la reutiliza; si no, usa la posición nueva.
57	`                return;`	Termina la inserción ya realizada. No sigue recorriendo ni añade otro registro.
58	`            }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
59	`            if (casilla.estacion == null) {`	Comprueba si la casilla existente es una marca de borrado.
60	`                if (primeraBorrada < 0) {`	Solo recuerda la primera marca; las siguientes no deben desplazarla.
61	`                    primeraBorrada = posicion;`	Guarda el índice de esa primera marca para usarlo luego.
62	`                }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
63	`            } else if (casilla.estacion.getCodigo().equals(estacion.getCodigo())) {`	Si no era marca, compara contenido de códigos con equals; no compara referencias de String con ==.
64	`                throw new IllegalArgumentException("El código ya existe: " + estacion.getCodigo());`	Rechaza la clave duplicada aunque haya visto una marca antes.
65	`            } else {`	Entra cuando la casilla está activa y tiene otro código.
66	`                registrarColision();`	Cuenta la colisión; la escritura todavía no sucede.
67	`            }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
68	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
69	`        if (primeraBorrada >= 0) {`	Después del ciclo completo, comprueba si quedó una marca disponible.
70	`            ocupar(primeraBorrada, estacion);`	Inserta en esa marca aunque no exista ninguna casilla nunca ocupada.
71	`            return;`	Termina sin lanzar el error de capacidad.

Línea	Código original	Qué hace y por qué
72	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
73	`        throw new IllegalStateException("No hay una casilla disponible; falta ampliar la tabla");`	Si no encontró posición, informa un límite del estado actual. No sobrescribe datos.
74	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
75		Separación visual; no ejecuta instrucciones.
76	`    /** Devuelve la misma instancia almacenada, o null si el código no existe. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
77	`    public Estacion buscar(String codigo) {`	Busca por código y devuelve una referencia a Estacion o null.
78	`        iniciarOperacion();`	Inicia la medición de esta búsqueda.
79	`        validarCodigo(codigo);`	Comprueba el formato antes de llamar al algoritmo interno.
80	`        int posicion = localizar(codigo);`	Obtiene el índice del código, o -1 si no está.
81	`        return posicion < 0 ? null : casillas[posicion].estacion;`	Evita acceder al índice -1. Si hay coincidencia, devuelve el mismo objeto almacenado.
82	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
83		Separación visual; no ejecuta instrucciones.
84	`    /** Mantiene la marca de borrado para no cortar las cadenas de búsqueda. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
85	`    public void eliminar(String codigo) {`	Elimina por código; el resultado se ve en el estado de la tabla.
86	`        iniciarOperacion();`	Empieza las mediciones de esta eliminación.
87	`        validarCodigo(codigo);`	Rechaza códigos nulos o mal formados antes de leer el arreglo.
88	`        int posicion = localizar(codigo);`	Reutiliza el recorrido de búsqueda en lugar de duplicarlo.
89	`        if (posicion < 0) {`	Distingue una eliminación inexistente.
90	`            throw new IllegalArgumentException("El código no existe: " + codigo);`	Informa ausencia y conserva los registros y el tamaño.
91	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
92	`        casillas[posicion].estacion = null;`	Borra la estación, pero conserva el objeto Casilla como marca.
93	`        tamanio--;`	Descuenta exactamente un registro activo; no modifica capacidad.
94	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
95		Separación visual; no ejecuta instrucciones.
96	`    private int localizar(String codigo) {`	Ayudante interno que devuelve un índice, no una Estacion.
97	`        int inicio = hash(codigo);`	Usa la misma función hash que insertar. Si no coincidieran, se perderían búsquedas.

Línea	Código original	Qué hace y por qué
98	`        for (int intento = 0; intento < casillas.length; intento++) {`	Acota la exploración a capacidad intentos.
99	`            int posicion = estrategia.posicion(inicio, intento, casillas.length);`	Usa la misma secuencia de posiciones que la inserción.
100	`            Casilla casilla = sondear(posicion);`	Toda lectura pasa por sondear para que las métricas sean coherentes.
101	`            if (casilla == null) {`	Si nunca se ocupó esta posición, la clave no puede estar más adelante en su cadena.
102	`                return -1;`	Devuelve la señal de ausencia sin recorrer innecesariamente el resto.
103	`            }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
104	`            if (casilla.estacion != null) {`	Solo compara claves de casillas activas. Una marca se atraviesa sin contar colisión.
105	`                if (casilla.estacion.getCodigo().equals(codigo)) {`	Comprueba igualdad de contenido del código buscado.
106	`                    return posicion;`	Devuelve el índice donde encontró la estación.
107	`                }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
108	`                registrarColision();`	La casilla activa tenía otra clave: cuenta una colisión y continúa.
109	`            }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
110	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
111	`        return -1;`	Si agotó los intentos sin coincidencia, también informa ausencia.
112	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
113		Separación visual; no ejecuta instrucciones.
114	`    /** Hash polinómico de toda la cadena; se reduce en cada paso. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
115	`    private int hash(String codigo) {`	Función interna que transforma un código en un índice válido.
116	`        int resultado = 0;`	Empieza la acumulación polinómica en cero.
117	`        for (int i = 0; i < codigo.length(); i++) {`	Recorre los caracteres de izquierda a derecha; sus posiciones importan.
118	`            resultado = (int) ((31L * resultado + codigo.charAt(i)) % casillas.length);`	Multiplica por 31 en long, suma el valor del carácter y reduce módulo capacidad; el resultado reducido cabe en int.
119	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
120	`        return resultado;`	Devuelve el índice inicial calculado.
121	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
122		Separación visual; no ejecuta instrucciones.

Línea	Código original	Qué hace y por qué
123	`    private static void validarCodigo(String codigo) {`	Validador común; static porque no necesita el estado de una tabla.
124	`        if (codigo == null || !codigo.matches("CBA-[0-9]{4}")) {`	El OR de cortocircuito evita llamar matches sobre null; la expresión exige cuatro dígitos ASCII.
125	`            throw new IllegalArgumentException("El código debe tener el formato CBA-NNNN");`	Rechaza códigos inválidos con la misma convención del modelo.
126	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
127	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
128		Separación visual; no ejecuta instrucciones.
129	`    private void ocupar(int posicion, Estacion estacion) {`	Ayudante que ocupa una posición ya elegida por insertar.
130	`        casillas[posicion] = new Casilla(estacion);`	Crea una casilla activa, reemplazando una referencia null o una marca previa.
131	`        tamanio++;`	Aumenta la cantidad de registros activos una sola vez por inserción aceptada.
132	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
133		Separación visual; no ejecuta instrucciones.
134	`    private Casilla sondear(int posicion) {`	Centraliza una lectura del arreglo y su medición.
135	`        sondeosUltimaOperacion++;`	Suma uno a los sondeos de la última operación.
136	`        sondeosTotales++;`	También suma uno al acumulado histórico.
137	`        return casillas[posicion];`	Devuelve el contenido de la posición, que puede ser null.
138	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
139		Separación visual; no ejecuta instrucciones.
140	`    private void registrarColision() {`	Centraliza la medición de una clave activa diferente.
141	`        colisionesUltimaOperacion++;`	Aumenta las colisiones de la operación actual.
142	`        colisionesTotales++;`	Aumenta el acumulado de colisiones.
143	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
144		Separación visual; no ejecuta instrucciones.
145	`    private void iniciarOperacion() {`	Preparación compartida por insertar, buscar y eliminar.
146	`        sondeosUltimaOperacion = 0;`	Borra el valor de sondeos de la operación anterior.
147	`        colisionesUltimaOperacion = 0;`	Borra el valor de colisiones anterior; conserva los totales.
148	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
149		Separación visual; no ejecuta instrucciones.
150	`    public int tamanio() {`	Consulta pública del tamaño; no sondea ni reinicia contadores.
151	`        return tamanio;`	Devuelve la cantidad activa mantenida por insertar y eliminar.

Línea	Código original	Qué hace y por qué
152	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
153		Separación visual; no ejecuta instrucciones.
154	`    public boolean estaVacia() {`	Permite preguntar si no hay estaciones activas.
155	`        return tamanio == 0;`	Devuelve el resultado booleano de comparar tamaño con cero; puede haber marcas aunque esté vacía.
156	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
157		Separación visual; no ejecuta instrucciones.
158	`    public int capacidad() {`	Consulta la capacidad física actual del arreglo.
159	`        return casillas.length;`	Devuelve su longitud, no la cantidad de estaciones.
160	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
161		Separación visual; no ejecuta instrucciones.
162	`    /** Ocupación de estaciones activas; no incluye marcas de borrado. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
163	`    public double factorCarga() {`	Calcula la proporción de ocupación activa.
164	`        return (double) tamanio / casillas.length;`	Convierte tamaño a double antes de dividir para evitar división entera truncada.
165	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
166		Separación visual; no ejecuta instrucciones.
167	`    public int getSondeosUltimaOperacion() {`	Getter de sondeos de la última operación.
168	`        return sondeosUltimaOperacion;`	Devuelve la medición sin cambiarla.
169	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
170		Separación visual; no ejecuta instrucciones.
171	`    public int getColisionesUltimaOperacion() {`	Getter de colisiones de la última operación.
172	`        return colisionesUltimaOperacion;`	Devuelve las colisiones recientes sin sumarlas otra vez.
173	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
174		Separación visual; no ejecuta instrucciones.
175	`    public long getSondeosTotales() {`	Getter del acumulado de sondeos, de tipo long.
176	`        return sondeosTotales;`	Devuelve el total desde el último reinicio.
177	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
178		Separación visual; no ejecuta instrucciones.
179	`    public long getColisionesTotales() {`	Getter del acumulado de colisiones.
180	`        return colisionesTotales;`	Devuelve ese total histórico.
181	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

Línea	Código original	Qué hace y por qué
182		Separación visual; no ejecuta instrucciones.
183	`    /** Reinicia las mediciones sin modificar estaciones ni capacidad. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
184	`    public void reiniciarContadores() {`	Reinicia todas las mediciones para empezar, por ejemplo, un lote de búsquedas.
185	`        iniciarOperacion();`	Reutiliza el reinicio de los contadores recientes.
186	`        sondeosTotales = 0;`	Pone a cero el acumulado de sondeos sin tocar el arreglo.
187	`        colisionesTotales = 0;`	Pone a cero el acumulado de colisiones sin cambiar tamaño ni datos.
188	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
189	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

189 líneas explicadas. SHA-256: 96ebc221f4439602530ed524e11fb699e828be319bc5eb12ec87e74493c80357

10 Práctica manual línea por línea

ejemplos\m1-manual.jsh

Cada fila corresponde a una línea física del archivo. Las líneas vacías, comentarios y llaves se conservan para que los números coincidan. Leé primero el código y después explicá su efecto sin mirar la tercera columna.

Línea	Código original	Qué hace y por qué
1	`import redsens.registro.TablaHash;`	Importa la tabla ya compilada para usar su nombre corto en JShell.
2	`import redsens.registro.EstrategiaExploracion;`	Importa el selector de estrategia.
3	`import redsens.modelo.Estacion;`	Importa el modelo de los compañeros.
4	`import redsens.modelo.TipoEstacion;`	Importa la enumeración de tipos válidos.
5		Separación visual; no ejecuta instrucciones.
6	`TablaHash tabla = new TablaHash(17, EstrategiaExploracion.LINEAL);`	Crea una tabla lineal vacía con la capacidad 17 del grupo.
7	`Estacion e0 = new Estacion("CBA-0000", 0, "Estación cero", "San Francisco", TipoEstacion.METEOROLOGICA, 80, 0);`	Construye la estación de código 0000, batería 80 y cero días sin visita.
8	`Estacion e13 = new Estacion("CBA-0013", 13, "Estación trece", "San Francisco", TipoEstacion.METEOROLOGICA, 15, 4);`	Construye la estación 0013, crítica porque batería 15 es menor que 20.
9	`Estacion e26 = new Estacion("CBA-0026", 26, "Estación veintiséis", "San Francisco", TipoEstacion.HIDROMETRICA, 60, 2);`	Construye la estación 0026 de tipo hidrométrica.
10		Separación visual; no ejecuta instrucciones.
11	`tabla.insertar(e0);`	Inserta 0000 en su hash inicial 12.
12	`tabla.insertar(e13);`	0013 colisiona en 12 y se guarda en 13.
13	`tabla.insertar(e26);`	0026 colisiona en 12 y 13; se guarda en 14.
14	`System.out.println("Inserción de 0026: " + tabla.getSondeosUltimaOperacion() + " sondeos, " + tabla.getColisionesUltimaOperacion() + " colisiones");`	Lee inmediatamente las mediciones de la tercera inserción: 3 sondeos y 2 colisiones.
15	`System.out.println("Acumulados: " + tabla.getSondeosTotales() + " sondeos, " + tabla.getColisionesTotales() + " colisiones");`	Muestra el acumulado de las tres inserciones: 6 y 3.
16	`System.out.println("Tamaño: " + tabla.tamanio() + "; capacidad: " + tabla.capacidad());`	Muestra registros activos y capacidad física por separado.
17		Separación visual; no ejecuta instrucciones.
18	`tabla.eliminar("CBA-0000");`	Borra 0000 y deja una marca en 12.
19	`System.out.println("Buscar 0026 tras borrar 0000: " + tabla.buscar("CBA-0026").getNombre());`	Busca 0026 atravesando la marca y muestra su nombre; en este caso sabemos que existe antes de llamar getNombre.
20	`System.out.println("Búsqueda de 0026: " + tabla.getSondeosUltimaOperacion() + " sondeos, " + tabla.getColisionesUltimaOperacion() + " colisiones");`	Muestra 3 sondeos y 1 colisión de la búsqueda anterior.
21	`System.out.println("Buscar 0000 borrada: " + tabla.buscar("CBA-0000"));`	Busca el código borrado y muestra null; esta búsqueda reemplaza los contadores recientes.
22		Separación visual; no ejecuta instrucciones.
23	`tabla.insertar(e0);`	Reinserta e0; recuerda la marca, comprueba duplicados y luego la reutiliza.

Línea	Código original	Qué hace y por qué
24	`System.out.println("Reinsertar 0000: " + tabla.getSondeosUltimaOperacion() + " sondeos, " + tabla.getColisionesUltimaOperacion() + " colisiones");`	Muestra los 4 sondeos y 2 colisiones de esa reinserción.
25	`System.out.println("Factor de carga: " + tabla.factorCarga());`	Muestra 3/17 calculado en punto flotante.
26	`tabla.reiniciarContadores();`	Reinicia las mediciones sin borrar los datos.
27	`System.out.println("Sondeos tras reiniciar: " + tabla.getSondeosTotales());`	Muestra cero en sondeos acumulados.
28		Separación visual; no ejecuta instrucciones.
29	`// La sesión sigue abierta para probar otras operaciones. Salir con /exit.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.

29 líneas explicadas. SHA-256: 175d6a7046c3490b9f64a986c434f0ce4acbe2bebde81342d22b7b83d4e093c2

11 PruebaTablaHash línea por línea

src\test\java\redsens\registro\PruebaTablaHash.java

Cada fila corresponde a una línea física del archivo. Las líneas vacías, comentarios y llaves se conservan para que los números coincidan. Leé primero el código y después explicá su efecto sin mirar la tercera columna.

Línea	Código original	Qué hace y por qué
1	`package redsens.registro;`	Las pruebas usan el mismo paquete de las clases del registro.
2		Separación visual; no ejecuta instrucciones.
3	`import java.util.Locale;`	Locale.ROOT permite formatear códigos sin depender de la configuración regional.
4	`import java.util.Random;`	Random aporta secuencias reproducibles para las pruebas.
5	`import redsens.modelo.Estacion;`	Importa el modelo compartido de estación.
6	`import redsens.modelo.TipoEstacion;`	Importa los tipos válidos del modelo.
7		Separación visual; no ejecuta instrucciones.
8	`/** Casos de M1 lineal ejecutables sin dependencias externas. */`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
9	`public final class PruebaTablaHash {`	Clase de pruebas ejecutable. final impide derivar de ella.
10	`    private static int verificaciones;`	Acumula las condiciones que pasaron. static comparte el valor entre métodos.
11		Separación visual; no ejecuta instrucciones.
12	`    public static void main(String[] args) {`	Punto de entrada que ejecuta java; no hace falta un framework de pruebas.
13	`        probarColisionesYBorrado();`	Ejecuta el caso de colisiones, marcas y contadores.
14	`        probarVueltaAlInicioYTablaLlena();`	Ejecuta vuelta al inicio, tabla llena y reutilización.
15	`        probarEntradasInvalidas();`	Ejecuta validaciones de entrada y capacidad.
16	`        probarOperacionesAleatorias();`	Compara operaciones mezcladas contra un arreglo de referencia independiente.
17	`        probarDiezMilEstaciones();`	Prueba integridad en todo el dominio de códigos.
18	`        System.out.println("Tabla hash lineal: " + verificaciones + " verificaciones OK");`	Imprime el total solo después de superar todos los casos anteriores.
19	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
20		Separación visual; no ejecuta instrucciones.
21	`    private static void probarColisionesYBorrado() {`	Agrupa un escenario pequeño cuyas posiciones se pueden anticipar a mano.
22	`        TablaHash tabla = new TablaHash(5, EstrategiaExploracion.LINEAL);`	Usa capacidad 5 para forzar colisiones fáciles; no es el parámetro del experimento oficial.
23	`        verificar(tabla.estaVacia() && tabla.tamanio() == 0, "Tabla vacía");`	Comprueba vacío y tamaño inicial coherentes.
24	`        verificar(tabla.buscar(codigo(0)) == null, "Ausente en tabla vacía");`	Busca en vacío y espera ausencia, sin excepción.
25	`        verificar(tabla.getSondeosUltimaOperacion() == 1, "Se sondea la casilla vacía");`	Verifica que leer una casilla nunca ocupada también es sondeo.

Línea	Código original	Qué hace y por qué
26	`        // Estas tres cadenas tienen hash inicial 0 módulo 5.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
27	`        tabla.reiniciarContadores();`	Excluye la búsqueda anterior del lote de inserciones.
28	`        Estacion primera = estacion(0);`	Guarda una referencia que luego podrá compararse por identidad.
29	`        Estacion tercera = estacion(14);`	Prepara una tercera clave que colisiona módulo 5.
30	`        tabla.insertar(primera);`	Inserta la primera clave en su posición inicial.
31	`        tabla.insertar(estacion(5));`	Inserta una segunda clave que debe desplazarse una posición.
32	`        tabla.insertar(tercera);`	Inserta la tercera clave de la misma cadena.
33	`        verificar(tabla.getSondeosUltimaOperacion() == 3, "Tres sondeos al insertar");`	Exige tres lecturas en esa tercera inserción.
34	`        verificar(tabla.getColisionesUltimaOperacion() == 2, "Dos colisiones al insertar");`	Exige dos casillas activas con claves diferentes.
35	`        verificar(tabla.getSondeosTotales() == 6 && tabla.getColisionesTotales() == 3,`	Empieza una comprobación conjunta: sondeos 1+2+3 y colisiones 0+1+2.
36	`                "Contadores acumulados de inserción");`	Completa la llamada de verificación con el mensaje que se usará si falla.
37	`        verificar(tabla.factorCarga() == 0.6, "Factor de carga activo");`	Comprueba que 3/5 se calcula como 0,6 y no como cero entero.
38	`        tabla.eliminar(codigo(0));`	Borra la primera clave de la cadena.
39	`        verificar(tabla.buscar(codigo(14)) == tercera, "La marca no corta la búsqueda");`	Comprueba que la estación posterior sigue accesible y es el mismo objeto.
40	`        verificar(tabla.getSondeosUltimaOperacion() == 3, "La marca también es un sondeo");`	La búsqueda atravesó tres posiciones, incluida la marca.
41	`        verificar(tabla.getColisionesUltimaOperacion() == 1, "La marca no es colisión");`	Solo la clave intermedia diferente cuenta como colisión.
42	`        esperar(IllegalArgumentException.class, () -> tabla.insertar(estacion(14)));`	La lambda intenta duplicar una clave detrás de la marca; se exige la excepción prevista.
43	`        verificar(tabla.tamanio() == 2, "No duplica una clave detrás de un borrado");`	Comprueba que el rechazo no agregó una estación extra.
44	`        tabla.insertar(primera);`	Reinserta la primera estación y reutiliza una marca.
45	`        verificar(tabla.buscar(codigo(0)) == primera, "Reutiliza la primera marca");`	La búsqueda por identidad confirma que volvió la referencia original.
46	`        verificar(tabla.getSondeosUltimaOperacion() == 1, "Reutilización en posición inicial");`	Buscarla en su posición inicial debe requerir un sondeo.
47	`        tabla.reiniciarContadores();`	Reinicia contadores con datos todavía presentes.
48	`        verificar(tabla.getSondeosTotales() == 0 && tabla.getColisionesTotales() == 0`	Empieza la comprobación de que los acumulados se reiniciaron.
49	`                && tabla.getSondeosUltimaOperacion() == 0, "Reinicio de mediciones");`	Agrega la condición de que también se reinició la última operación.
50	`        verificar(tabla.tamanio() == 3, "El reinicio conserva los datos");`	Comprueba que medir no haya borrado los tres registros.
51	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

Línea	Código original	Qué hace y por qué
52		Separación visual; no ejecuta instrucciones.
53	`    private static void probarVueltaAlInicioYTablaLlena() {`	Agrupa casos que podrían producir ciclos infinitos si el algoritmo no estuviera acotado.
54	`        TablaHash tabla = new TablaHash(5);`	Usa el constructor abreviado que elige LINEAL.
55	`        // Hash 4: la cadena de exploración cruza del final al comienzo.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
56	`        tabla.insertar(estacion(4));`	Primera clave con hash 4 módulo 5, al final del arreglo.
57	`        Estacion segunda = estacion(9);`	Guarda otra clave con igual posición inicial.
58	`        tabla.insertar(segunda);`	La segunda inserción debe volver del índice 4 al 0.
59	`        verificar(tabla.buscar(codigo(9)) == segunda, "Exploración circular");`	Confirma que la búsqueda reproduce esa vuelta circular.
60	`        tabla.insertar(estacion(18));`	Continúa llenando la cadena de colisiones.
61	`        tabla.insertar(estacion(27));`	Agrega la cuarta estación.
62	`        tabla.insertar(estacion(36));`	Ocupa la última casilla disponible.
63	`        esperar(IllegalStateException.class, () -> tabla.insertar(estacion(45)));`	Verifica fallo controlado al insertar una sexta estación en cinco lugares.
64	`        verificar(tabla.tamanio() == 5, "Fallo por capacidad conserva el tamaño");`	El fallo no puede alterar tamaño.
65	`        verificar(tabla.getSondeosUltimaOperacion() == 5, "El ciclo completo está acotado");`	La operación debe terminar tras cinco lecturas, no ciclar indefinidamente.
66	`        verificar(tabla.buscar(codigo(45)) == null, "Ausente sin casillas nunca ocupadas");`	Una búsqueda fallida también debe terminar con la tabla completamente ocupada.
67	`        verificar(tabla.getSondeosUltimaOperacion() == 5, "Búsqueda acotada en tabla llena");`	Exige un ciclo de cinco sondeos para esa búsqueda.
68	`        esperar(IllegalArgumentException.class, () -> tabla.insertar(estacion(9)));`	Duplicar una clave existente debe distinguirse del error de capacidad.
69	`        tabla.eliminar(codigo(18));`	Crea una marca dentro de la tabla llena.
70	`        tabla.insertar(estacion(45));`	Inserta otra clave aprovechando esa marca.
71	`        verificar(tabla.tamanio() == 5 && tabla.buscar(codigo(45)) != null,`	Comprueba tamaño lleno y accesibilidad del nuevo registro.
72	`                "Reutiliza un borrado sin casillas nuevas");`	Termina la verificación iniciada en la línea anterior.
73	`        for (int numero : new int[] {4, 9, 27, 36, 45}) {`	Recorre las cinco claves activas mediante un arreglo literal.
74	`            tabla.eliminar(codigo(numero));`	Elimina cada registro para dejar todas las posiciones como marcas.
75	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
76	`        verificar(tabla.estaVacia(), "Todas las casillas son marcas de borrado");`	Aunque quedan casillas, no hay estaciones activas.
77	`        verificar(tabla.buscar(codigo(99)) == null, "Termina con todas las casillas borradas");`	Buscar debe terminar aun sin encontrar una posición nunca ocupada.
78	`        tabla.insertar(estacion(99));`	Reinserta en un arreglo compuesto solo por marcas.

Línea	Código original	Qué hace y por qué
79	`verificar(tabla.tamanio() == 1, "Inserta con todas las casillas borradas");`	Verifica que la nueva inserción se contó una vez.
80	`TablaHash unica = new TablaHash(1);`	Prepara el extremo de una sola casilla.
81	`unica.insertar(estacion(0));`	Ocupa esa única posición.
82	`unica.eliminar(codigo(0));`	La convierte en marca de borrado.
83	`unica.insertar(estacion(1));`	Reutiliza la misma casilla con otra clave.
84	`verificar(unica.buscar(codigo(1)) != null, "Capacidad mínima de una casilla");`	Comprueba que capacidad uno también funciona.
85	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
86		Separación visual; no ejecuta instrucciones.
87	`private static void probarEntradasInvalidas() {`	Agrupa casos donde los argumentos deben rechazarse explícitamente.
88	`esperar(IllegalArgumentException.class, () -> new TablaHash(0));`	Capacidad cero es inválida.
89	`esperar(IllegalArgumentException.class, () -> new TablaHash(-1));`	Capacidad negativa es inválida.
90	`esperar(IllegalArgumentException.class, () -> new TablaHash(17, null));`	La estrategia no puede ser null.
91	`TablaHash tabla = new TablaHash(17);`	Crea una tabla válida de capacidad 17 para los siguientes casos.
92	`esperar(IllegalArgumentException.class, () -> tabla.insertar(null));`	Insertar null debe fallar antes de intentar obtener su código.
93	`for (String invalido : new String[] {null, "", "CBA-12", "CBA-10000", "ABC-0000"}) {`	Recorre una colección de códigos nulos o mal formados.
94	`esperar(IllegalArgumentException.class, () -> tabla.buscar(invalido));`	Cada búsqueda inválida debe lanzar IllegalArgumentException.
95	`esperar(IllegalArgumentException.class, () -> tabla.eliminar(invalido));`	Eliminar aplica la misma validación del código.
96	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
97	`esperar(IllegalArgumentException.class, () -> tabla.eliminar(codigo(0)));`	Una eliminación con código válido pero ausente también se rechaza.
98	`verificar(tabla.estaVacia(), "Entradas inválidas no alteran la tabla");`	Las entradas inválidas no deben haber insertado datos.
99	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
100		Separación visual; no ejecuta instrucciones.
101	`private static void probarOperacionesAleatorias() {`	Prueba de operaciones mezcladas para descubrir interacciones entre borrado e inserción.
102	`TablaHash tabla = new TablaHash(211);`	Capacidad 211 para un dominio de solo 150 claves; no se espera agotar capacidad.
103	`Estacion[] referencia = new Estacion[150];`	Referencia independiente: cada número tiene un lugar directo; no usa una tabla de biblioteca como sustituto de M1.
104	`Random random = new Random(3026L);`	Fija la semilla asignada al grupo para repetir la prueba.

Línea	Código original	Qué hace y por qué
105	`        int cantidad = 0;`	Lleva por separado la cantidad esperada de registros.
106	`        for (int i = 0; i < 3000; i++) {`	Repite 3.000 operaciones.
107	`            int numero = random.nextInt(referencia.length);`	Elige una clave del dominio 0..149.
108	`            int operacion = random.nextInt(3);`	Elige insertar, eliminar o buscar con valores 0, 1 o 2.
109	`            if (operacion == 0) {`	Selecciona el caso de inserción.
110	`                if (referencia[numero] == null) {`	Consulta la referencia para saber si la inserción debe aceptarse.
111	`                    referencia[numero] = estacion(numero);`	Crea el objeto esperado en la referencia.
112	`                    tabla.insertar(referencia[numero]);`	Inserta exactamente esa misma instancia en la tabla.
113	`                    cantidad++;`	Actualiza el número esperado de activos.
114	`                } else {`	Alternativa del caso de inserción: la clave ya existía.
115	`                    esperar(IllegalArgumentException.class, () -> tabla.insertar(estacion(numero)));`	Comprueba que la tabla rechaza ese duplicado.
116	`                }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
117	`            } else if (operacion == 1) {`	Selecciona el caso de eliminación.
118	`                if (referencia[numero] == null) {`	Comprueba si la clave está ausente en la referencia.
119	`                    esperar(IllegalArgumentException.class, () -> tabla.eliminar(codigo(numero)));`	Exige rechazo cuando se intenta eliminar un registro ausente.
120	`                } else {`	Alternativa: sí existe una estación para eliminar.
121	`                    tabla.eliminar(codigo(numero));`	La elimina de la tabla bajo prueba.
122	`                    referencia[numero] = null;`	Borra la referencia esperada.
123	`                    cantidad--;`	Descuenta un activo esperado.
124	`                }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
125	`            } else {`	Tercer caso de operación: búsqueda.
126	`                verificar(tabla.buscar(codigo(numero)) == referencia[numero], "Búsqueda contra referencia");`	Compara referencia a referencia: misma estación o null en ambas.
127	`            }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
128	`            verificar(tabla.tamanio() == cantidad, "Cantidad tras operación aleatoria");`	Después de cada operación comprueba que el tamaño no se desvió.
129	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
130	`        for (int numero = 0; numero < referencia.length; numero++) {`	Recorre todo el dominio al terminar las operaciones aleatorias.
131	`            verificar(tabla.buscar(codigo(numero)) == referencia[numero], "Estado final contra referencia");`	Comprueba el contenido completo final contra la referencia.
132	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
133	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

Línea	Código original	Qué hace y por qué
134		Separación visual; no ejecuta instrucciones.
135	`    private static void probarDiezMilEstaciones() {`	Prueba de integridad con los 10.000 códigos válidos.
136	`        // Capacidad fija suficiente: esto es una prueba de integridad, no el experimento M1.`	Comentario de documentación: explica el contrato o el caso, pero no se ejecuta.
137	`        TablaHash tabla = new TablaHash(20011);`	Reserva capacidad fija suficiente; todavía no prueba rehashing desde 17.
138	`        int[] orden = new int[10000];`	Arreglo de números que luego se mezclará.
139	`        Estacion[] referencia = new Estacion[10000];`	Arreglo con las instancias esperadas para las búsquedas finales.
140	`        for (int i = 0; i < orden.length; i++) {`	Recorre todo el dominio de códigos.
141	`            orden[i] = i;`	Inicializa el orden ascendente.
142	`            referencia[i] = estacion(i);`	Crea una estación válida por cada número.
143	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
144	`        Random random = new Random(3026L);`	Fija una semilla reproducible para mezclar el orden de inserción.
145	`        for (int i = orden.length - 1; i > 0; i--) {`	Mezcla Fisher-Yates: recorre desde el final hasta el índice uno.
146	`            int j = random.nextInt(i + 1);`	Elige uniformemente una posición entre cero e i, inclusive.
147	`            int aux = orden[i];`	Guarda temporalmente el valor para no perderlo al intercambiar.
148	`            orden[i] = orden[j];`	Mueve el valor elegido a la posición actual.
149	`            orden[j] = aux;`	Completa el intercambio en la otra posición.
150	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
151	`        for (int numero : orden) {`	Recorre las claves en el orden ya mezclado.
152	`            tabla.insertar(referencia[numero]);`	Inserta cada objeto esperado exactamente una vez.
153	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
154	`        verificar(tabla.tamanio() == 10000, "Diez mil códigos únicos");`	Comprueba que no faltan ni se duplicaron registros.
155	`        for (int numero = 0; numero < referencia.length; numero++) {`	Recorre nuevamente todos los códigos válidos.
156	`            verificar(tabla.buscar(codigo(numero)) == referencia[numero], "Integridad de diez mil estaciones");`	Cada búsqueda debe devolver la misma instancia guardada al construir los datos.
157	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
158	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
159		Separación visual; no ejecuta instrucciones.
160	`    private static String codigo(int numero) {`	Ayudante que transforma un entero en código válido.

Línea	Código original	Qué hace y por qué
161	`        return String.format(Locale.ROOT, "CBA-%04d", numero);`	%04d formatea cuatro dígitos con ceros a la izquierda; Locale.ROOT evita variaciones regionales.
162	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
163		Separación visual; no ejecuta instrucciones.
164	`    private static Estacion estacion(int numero) {`	Fábrica pequeña de datos de prueba; no reemplaza el generador M6.
165	`        return new Estacion(codigo(numero), numero, "Estación " + numero, "San Francisco",`	Comienza la creación de Estacion con código y número coherentes y una localidad del grafo.
166	`                TipoEstacion.METEOROLOGICA, numero % 101, 0);`	Completa tipo, batería entre 0 y 100 y cero días sin visita.
167	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
168		Separación visual; no ejecuta instrucciones.
169	`    private static void verificar(boolean condicion, String mensaje) {`	Comprobación propia que sustituye un framework de pruebas.
170	`        if (!condicion) {`	Solo entra cuando la condición esperada no se cumple.
171	`            throw new AssertionError(mensaje);`	Interrumpe la ejecución con un error descriptivo; no se imprimirá un falso resumen de éxito.
172	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
173	`        verificaciones++;`	Cuenta una condición correcta.
174	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
175		Separación visual; no ejecuta instrucciones.
176	`    private static void esperar(Class<? extends RuntimeException> tipo, Runnable accion) {`	Recibe el tipo de excepción esperado y una acción diferida. La lambda aún no se ejecutó al pasarla.
177	`        try {`	Abre el bloque donde se ejecutará la acción que puede fallar.
178	`            accion.run();`	Ejecuta la acción mediante Runnable.run.
179	`        } catch (RuntimeException e) {`	Captura RuntimeException para revisar su tipo; no captura AssertionError.
180	`            verificar(tipo.isInstance(e), "Tipo de excepción: " + e);`	Comprueba compatibilidad del tipo real con el tipo esperado.
181	`            return;`	Si recibió la excepción correcta, termina la comprobación.
182	`        }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
183	`        throw new AssertionError("Se esperaba " + tipo.getSimpleName());`	Si la acción no lanzó excepción, la prueba falla explícitamente.
184	`    }`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.
185	`}`	Cierra el bloque abierto correspondiente: condición, ciclo, método o clase según su sangría.

185 líneas explicadas. SHA-256: 86d49c2ebc5cd64a95cdd55f9bdad6b1217f6cab1f22700fc803a5ea8baef63e

12 Preguntas para la exposición

¿Qué estructura elegiste y por qué?

Una tabla hash propia con direccionamiento abierto y exploración lineal, porque M1 requiere acceso por código en tiempo constante promedio y comparar estrategias de colisión. El arreglo concentra las casillas y permite una implementación propia explicable.

¿Hash igual significa estaciones iguales?

No. Dos códigos diferentes pueden tener igual posición inicial. Solo equals sobre el código confirma que es la misma clave.

¿Por qué no se borra poniendo casillas[i] = null?

Porque null significa nunca ocupada y detiene la búsqueda. Una marca conserva la continuidad de la cadena de exploración.

¿Por qué no se inserta apenas se ve una marca?

Porque el código puede existir más adelante. Se recuerda la primera marca y se sigue para evitar duplicados.

¿Cuándo termina una búsqueda?

Cuando encuentra la clave, llega a una casilla nunca ocupada o agota capacidad intentos. No termina por encontrar una marca.

¿Cuánto tarda buscar?

O(1) en promedio con carga y dispersión adecuadas, O(m) en el peor caso. No prometo O(1) garantizado. Muchos borrados también pueden aumentar sondeos aunque baje la carga activa.

¿Qué cambia al usar cuadrática?

La secuencia de posiciones. Se mantienen el modelo, las operaciones y los estados. Hay que justificar cobertura, capacidad prima y carga; i*i no visita todas las posiciones.

¿Qué relación hay con ABB y AVL?

Son índices distintos sobre los mismos objetos. La hash busca por codigo; los árboles ordenan por numero y permiten rangos y críticas. La integración debe mantener ambos coherentes.

¿Por qué no usar el número como índice de un arreglo?

Es una alternativa viable en este dominio fijo, pero no cumple la consigna de implementar y comparar una tabla hash propia. Su memoria depende del dominio reservado.

¿Qué significa que una estación sea inmutable?

Los campos del modelo no se cambian mediante setters. La tabla guarda referencias y puede borrarlas, pero no modifica batería ni nombre del objeto. Una futura actualización debe coordinar sustituciones en los índices.

¿M1 ya está completo?

Está completa nuestra parte lineal validada. Faltan cuadrática, umbrales, rehashing, experimento comparativo e integración. La prueba de 10.000 usa capacidad fija 20.011 y no demuestra crecimiento desde 17.

¿Qué modificó o revisó la persona y qué hizo la IA?

Codex implementó y documentó esta entrega. La revisión y comprensión humana deben registrarse cuando ocurran. No hay que atribuir autoría manual ni inventar un error detectado para la bitácora.

13 Cómo estudiar y verificar comprensión

Primero hacé la traza 12, 13, 14 con lápiz y papel. Explicá qué cambia al borrar la primera clave. Después ejecutá la práctica y compará tus predicciones con los resultados. Si no coinciden, mirá qué operación fue la última antes de leer los contadores.

Luego estudiá insertar y localizar en paralelo: ambos deben seguir las mismas posiciones. Marcá las tres salidas de insertar: casilla nueva, marca después del ciclo o error por capacidad. En localizar marcá las salidas: coincidencia, casilla nunca ocupada y fin de intentos.

Por último explicá el código a otra persona sin leer las notas. Practicá un cambio chico: usar otra capacidad en una tabla de prueba, eliminar una clave intermedia o comentar temporalmente una marca en una copia local. Antes de publicar cambios, restaurá el comportamiento correcto y ejecutá las pruebas.

Lista de comprobación personal

Puedo distinguir tamaño y capacidad.
Puedo calcular el siguiente índice con módulo.
Puedo diferenciar casilla null de estación null.
Puedo explicar por qué se rechaza un duplicado detrás de una marca.
Puedo decir qué cuenta como sondeo y colisión.
Puedo ejecutar M1 sin el menú general.
Puedo explicar qué necesita el compañero para cuadrática y rehashing.
Puedo describir una operación con peor caso lineal sin prometer rendimiento garantizado.

Fuentes y alcance de esta guía

Corrección real para la bitácora de IA: un borrador de esta guía decía que las tres claves de la traza tenían hash inicial 16 módulo 17. Codex comprobó la fórmula y obtuvo 12 para las tres; corrigió las posiciones a 12, 13 y 14 antes de publicar. El código Java y sus métricas no cambiaron. Es una validación asistida; no se atribuye esa revisión a FEDE.

TrabajoPracticoIntegradorEnunciado.pdf: modelo y restricciones, páginas 2 y 3; M1 y experimento, página 3; integración y parámetros, página 6; defensa oral y uso de IA, páginas 7 y 8. El roadmap tp ayed2.docx y su versión compartida establecen el reparto. Se usa como fuente principal del comportamiento el código de esta entrega y sus pruebas ejecutadas.

Código compartido: https://github.com/benjaminbogado/RedSens/tree/codex/tabla-hash-lineal
Contrato de continuación: resultados/contrato-m1-lineal.md
Práctica: ejemplos/m1-manual.jsh
Compilación y comandos: README.md

La guía no certifica comprensión individual ni aprobación de un docente. El material sirve para que puedas alcanzar esa comprensión practicando las operaciones y sus decisiones concretas.